Лабораторная работа №5.
Методология CI/CD

Содержание

Содержание

1 Цели лабораторной работы **3**

2 Задачи к лабораторной работе **3**

3 Методические материалы **4**

3.1 Что такое CI/CD? 4

3.2 Непрерывная интеграция 4

3.3 Непрерывная доставка 5

3.4 Непрерывное развёртывание 5

3.5 GitHub Actions 5

4 Ход работы **10**

4.1 Подготовка 10

4.2 Покрытие тестами 10

4.3 Создание пайплайна 10

4.4 Создание переменных Actions 12

4.5 Проверка пайплайна 12

5 Контрольные вопросы **14**

1 Цели лабораторной работы

- Познакомиться с методологией CI/CD;
- Научиться работать с инструментом GitHub Actions.

2 Задачи к лабораторной работе

1. Изучить методические материалы к лабораторной работе;
2. Написать тесты для приложения на Python;
3. Написать пайплайн сборки, тестирования и загрузки образа приложения на Docker Hub.

3 Методические материалы

3.1 Что такое CI/CD?

CI/CD - одна из DevOps-методик, при которой стадии разработки продукта полностью или частично автоматизированы. Призвана оптимизировать и ускорить жизненный цикл разработки программного обеспечения.

Почему CI/CD важен? CI/CD помогает организациям избежать ошибок и сбоев в коде, поддерживая непрерывный цикл разработки и обновления программного обеспечения. Поскольку CI/CD автоматизирует ручное вмешательство человека, время простоя сводится к минимуму, а релизы кода происходят быстрее, снижается человеческий фактор.

3.2 Непрерывная интеграция

CI (Continuous Integration) - процесс автоматизации для разработчиков, который позволяет чаще объединять изменения кода в общую ветвь. По мере внесения обновлений запускаются автоматизированные этапы тестирования для обеспечения надежности слитых изменений кода.

CI можно рассматривать как решение проблемы одновременной разработки слишком большого количества ветвей приложения, которые могут конфликтовать друг с другом.

Успешный CI означает, что после объединения изменений, внесенных разработчиками в приложение, эти изменения проверяются путем автоматической сборки приложения и проведения различных уровней автоматизированного тестирования, как правило, модульных и интеграционных тестов, чтобы убедиться, что изменения не нарушили работу приложения. Одно из преимуществ CI заключается в том,

что если автоматизированное тестирование обнаруживает конфликт между новым и существующим кодом, то такие ошибки легче и быстрее исправить.

3.3 Непрерывная доставка

CD (Continuous Delivery) - непрерывная доставка автоматизирует выпуск проверенного кода в репозиторий (например, GitHub или реестр контейнеров) после сборки и тестирования приложения на этапе CI. Поэтому, чтобы процесс непрерывной доставки был эффективным, важно, чтобы непрерывная интеграция уже была встроена в ваш конвейер разработки.

Таким образом, цель непрерывной доставки заключается в том, чтобы иметь кодовую базу, которая всегда готова к развертыванию в производственной среде, и гарантировать, что это развертывание потребует минимальных усилий.

3.4 Непрерывное развёртывание

CD (Continuous Deploy) - заключительный этап конвейера CI/CD. Непрерывное развертывание является расширением непрерывной доставки и может означать автоматизацию выпуска изменений из репозитория в производство, где они могут быть использованы клиентами.

На практике непрерывное развертывание означает, что изменения, внесенные разработчиком, автоматически появятся в производственной среде (при условии, что они прошли автоматизированное тестирование).

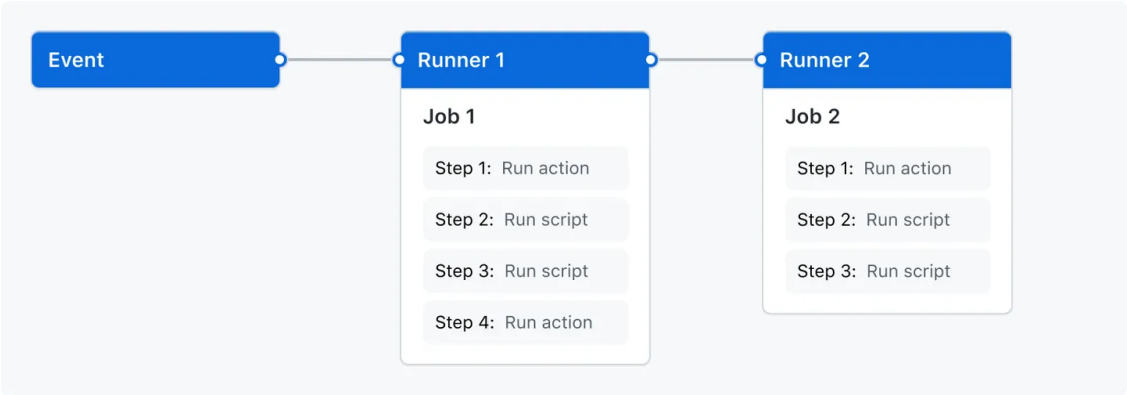
3.5 GitHub Actions

Существует ряд инструментов, реализующих CI/CD подход. Одним из них является GitHub Actions.

Из чего состоит GitHub Actions? В терминалогии GitHub Actions существуют следующие понятия:

- **рабочий процесс (workflow)** - это сценарий действий, который будет выполнен при наступлении определённых условий;
- рабочий процесс состоит из **заданий (jobs)**, которые могут выполняться в последовательном или параллельном порядке;

- каждое задание запускается внутри собственной виртуальной машины **исполнителя (runner)** или контейнера;
- задания состоят из одного или нескольких **шагов (steps)**;
- шаги либо выполняют определенный вами сценарий, либо запускают **действие (action)**, которое является многоразовым расширением, упрощающим рабочий процесс.



Workflows. Рабочий процесс - это настраиваемый автоматизированный процесс, который запускает одно или несколько **заданий**. Рабочие процессы определяются YAML-файлом и запускаются при наступлении события в хранилище, а также могут быть запущены вручную или по определенному расписанию.

Рабочие процессы определяются в каталоге **.github/workflows** в репозитории. В репозитории может быть несколько рабочих процессов, каждый из которых может выполнять различный набор задач. Вы можете ссылаться на рабочий процесс внутри другого рабочего процесса.

Events. Событие - это определенное действие в репозитории, которое запускает рабочий процесс. Например, событие может исходить от GitHub, когда кто-то создает pull request, открывает проблему или пушит коммит в репозиторий. Вы также можете запустить рабочий процесс по расписанию, отправив REST API запрос или вручную.

Jobs. Задание - это набор **шагов** в рабочем процессе, которые выполняются на одном и том же **исполнителе**. Каждый шаг - это либо сценарий оболочки, либо

действие, которые будут выполнены. Шаги выполняются по порядку и зависят друг от друга. Поскольку каждый шаг выполняется на одном и том же исполнителе, вы можете передавать данные от одного шага к другому. Например, у вас может быть этап, на котором собирается приложение, а затем этап, на котором оно тестируется.

По умолчанию задания не имеют зависимостей и выполняются параллельно. Вы можете указать зависимость одного задания от другого. В таком случае задание будет ожидать завершения выполнения того, от которого оно зависит.

Например, вы можете настроить несколько заданий для сборки приложения под разные архитектуры, а также задание упаковки, которое зависит от этих сборок. Задания сборки выполняются параллельно, и после их успешного завершения запускается задание упаковки.

Actions. **Действие** - это пользовательское приложение для платформы GitHub Actions, которое выполняет комплексную, часто повторяющуюся задачу. Действия используются, чтобы сократить количество повторяющегося кода в файлах рабочего процесса. Действие может извлечь ваш Git-репозиторий из GitHub, установить правильные инструменты для среды сборки или настроить аутентификацию для облачного провайдера.

Runners. **Исполнитель** - это сервер, который запускает рабочие процессы, когда они срабатывают. Каждый исполнитель может выполнять одно **задание** за раз. GitHub предоставляет исполнителей для Ubuntu Linux, Microsoft Windows и macOS для запуска **рабочих процессов**.

```
name: Test Python app
on:
  push:
    branches: ['dev']

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
    - name: Checkout
      uses: actions/checkout@v4
```

```
  - name: Setup Python
    uses: actions/setup-python@v5
    with:
      python-version: '3.11'

  - name: Install Dependencies
    run: |
      python -m pip install --upgrade pip
      pip install -r requirements.txt pytest

  - name: Run Tests
    run: python -m pytest tests/
```

Разберём этот пример:

- `name: Test Python app` - задаёт имя процессу;
- `on:` - ключевое слово для обозначения события, при котором запуститься процесс;
- `on.push:` - указывает на то, что процесс сработает при пуше;
- `on.push.branches:` - задаёт список веток, для которых отслеживается событие. По умолчанию - все ветки.
- `jobs:` - список заданий;
- `jobs.test:` - имя задания;
- `jobs.<job_id>.runs-on: ubuntu-latest` - определить тип машины для запуска;
- `jobs.<job_id>.steps:` - список шагов;
- `jobs.<job_id>.steps[*].name:` - имя шага;
- `jobs.<job_id>.steps[*].uses:` - ключевое слово для указания использования **действия** (в нашем случае: `actions/checkout@v4` и `actions/setup-python@v5`);

- `jobs.<job_id>.steps[*].with:` - перечень переменных, которые определены **действием** и необходимы для его работы;
- `jobs.<job_id>.steps[*].run:` - выполнение shell-команды, переданной в качестве значения.

Как было сказано ранее, по умолчанию задания запускаются параллельно. Для изменения этого поведения необходимо использовать ключевое слово `jobs.<job_id>.needs`.

```
jobs:
  job1:
    # do smth
  job2:
    needs: job1
    # do smth
```

В данном примере задание `job2` начнёт выполняться после успешного завершения `job1`.

Для установления зависимости между рабочими процессами используется `on.workflow_run.<branches|branches-ignore>`.

Например, рабочий процесс со следующим триггером будет запущен только тогда, когда рабочий процесс с именем `Build` будет запущен в ветке, имя которой начинается с `releases/`:

```
on:
  workflow_run:
    workflows: ["Build"]
    types: [requested]
    branches:
    - 'releases/**'
```

У триггера `workflow_run` есть три типа:

- `requested` - рабочий процесс был запущен;
- `in_progress` - событие возникает при начале обработки рабочего процесса на исполнителе;
- `completed` - событие возникает при завершении рабочего процесса, независимо от того, был ли он успешным или нет.

4 Ход работы

Пример файла workflow.

4.1 Подготовка

- Зарегистрируйтесь на Docker Hub;
- Склонируйте репозиторий;
- Создайте новый репозиторий на GitHub;
- В терминале перейдите в каталог склонированного репозитория и выполните команду, вставив вместо `<link-to-repository>` ссылку на созданный репозиторий:

  ```
  git remote set-url origin <link-to-repository>
  ```

- Создайте и перейдите на ветку `dev`.

4.2 Покрытие тестами

В файле `tests/test_user.py` находится пример теста на получение существующего пользователя `test_get_existed_user()`, а также объявлены ещё четыре метода, которые вам необходимо реализовать.

Чтобы узнать какие данные и статус-коды может возвращать приложение, изучите файл `src/routers/user.py`

4.3 Создание пайплайна

Создайте в корне проекта директорию `.github/workflows`, а в ней два файла:

1. `tests.yml`, который будет запускаться при любом коммите для запуска unit-тестов `tests`;
2. `build-and-delivery.yml`, который будет запускаться только при пуше в ветку `main` для сборки образа и доставки его в репозиторий Docker Hub.

tests.yml

- Задайте имя `Test Python App` для рабочего процесса;
- Задайте правило, при котором рабочий процесс будет запускаться при любом пуше в репозиторий;
- Определите секцию `jobs`, состоящую из одного задания `ci`:
  - Укажите `ubuntu-latest` в качестве типа машины для запуска;
  - Определите секцию `steps`:
    * Шаг для получения репозитория, используя действие `actions/checkout@v4`;
    * Шаг для настройки Python, используя действие `actions/setup-python@v5`. Также укажите версию Python `3.12`;
    * Шаг установки зависимостей приложения и библиотек для тестирования (`pytest` и `httpx`). Сначала установите `pip` с помощью `python -m pip install -upgrade pip`, а затем сами зависимости;
    * Шаг с запуском тестов `python -m pytest tests/`

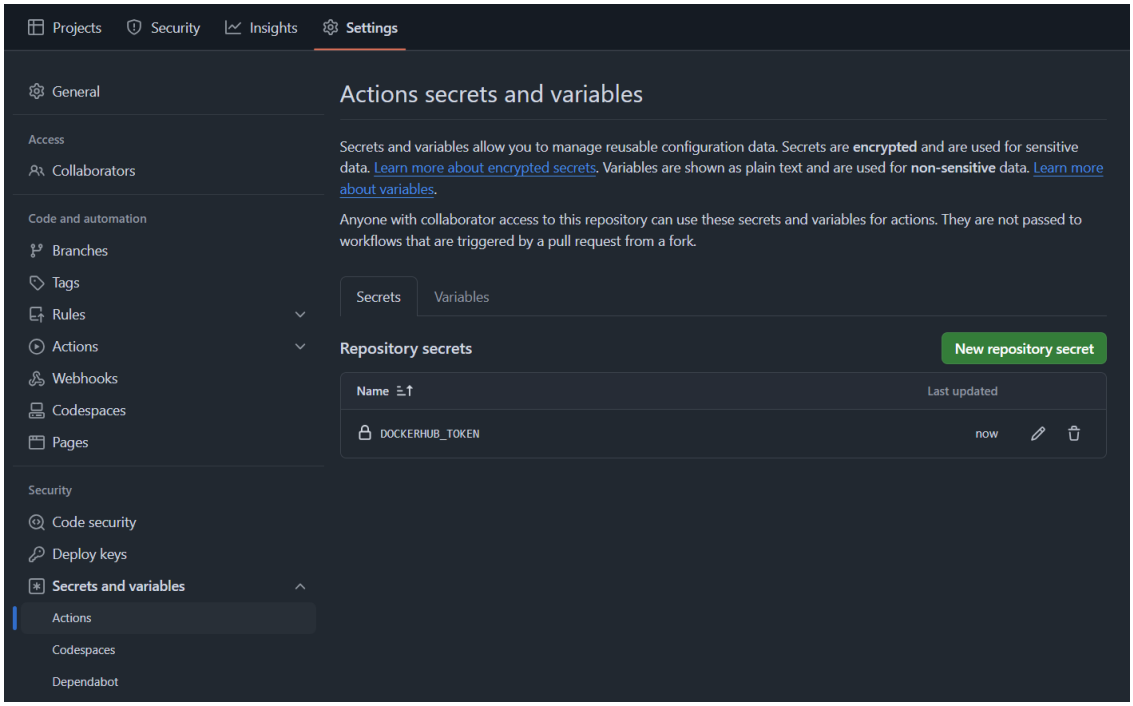
build-and-delivery.yml

- Задайте имя `Build and Delivery` для рабочего процесса;
- Задайте правило, при котором рабочий процесс будет запускаться только после завершения рабочего процесса `Test Python App` в ветке `main`;
- Определите секцию `jobs`, состоящую из одного задания `cd`:
  - Укажите `ubuntu-latest` в качестве типа машины для запуска;
  - Добавьте условие `if: ${{ github.event.workflow_run.conclusion == 'success' }}` для гарантии того, что это задание запустится только при успешном завершении предыдущего рабочего процесса;
  - Определите секцию `steps`:

* Шаг для логина на Docker Hub с использованием `ocker/login-action@v3`. Для этого действия задайте переменные `username: ${{ vars.DOCKERHUB_USERNAME }}` и `password: ${{ secrets.DOCKERHUB_TOKEN }}`;
* Шаг сборки и публикации образа на Docker Hub, используя `docker/build-push-action@v6`. Задайте переменные `push: true` и `tags: ${{ vars.DOCKERHUB_USERNAME }}/my-app:latest`

4.4 Создание переменных Actions

`vars.DOCKERHUB_USERNAME` и `secrets.DOCKERHUB_TOKEN` переменная и секрет GitHub Actions. Для их создания перейдите в настройки репозитория → раздел «Security» → «Secrets and variables» → «Actions».



Создайте переменную `DOCKERHUB_USERNAME` со значением имени пользователя Docker Hub и секрет `DOCKERHUB_TOKEN` со значением пароля от учётной записи.

4.5 Проверка пайплайна

После реализации тестов и создания рабочих процессов запушьте изменения в удалённый репозиторий в ветку `dev`. Перейдите на вкладку «Actions» и дождитесь завершения тестов.

При успешном завершении произведите слияние ветки `dev` в ветку `main` локально и запушьте изменения в удалённую ветку `main`. После тестирования должен запуститься процесс сборки и публикации образа. По его завершению на вашей странице Docker Hub должен появиться образ `DOCKERHUB_USERNAME/my-app` с тегом `latest`.

Теперь вы можете запустить контейнер Docker с использованием этого образа (тег `latest` используется по умолчанию):

```
$ docker run <DOCKERHUB_USERNAME>/my-app
```

5 Контрольные вопросы

1. Как CI/CD помогает в разработке?

2. К какому этапу CI/CD относится сборка десктоп приложения под операционные системы Windows, MacOS и Linux для его дальнейшего тестирования?

3. Что такое Continuous Delivery и Deploy?